

UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA
FACULTAD DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS
ESCUELA PROFESIONAL DE INGENIERÍA EN CIBERSEGURIDAD



Título:

**SISTEMA DE GESTIÓN DE EMPLEADOS MEDIANTE PROGRAMACIÓN
ORIENTADA A OBJETOS EN C++**

Asignatura:

ESTRUCTURA DE DATOS Y ALGORITMOS

Docente:

Mg. CARLOS ABRAHAM RIOS RIVERA

Estudiante:

CONDOR POMA, MIGUEL ANGEL

Tingo María – Perú

2026

RESUMEN

En esta práctica creamos un sistema para gestionar empleados con el lenguaje C++ y los principios de la Programación Orientada a Objetos. Nuestra aplicación nos permite administrar una lista con diez empleados, que se dividen en dos grupos: empleados que trabajan por hora y empleados fijos. Para cada uno de ellos, calculamos el salario según su cargo, como Gerente, Operador o Supervisor, y las horas que trabaja. Además, el sistema encuentra al empleado con el salario más alto, identifica a los trabajadores con más antigüedad en la empresa y organiza los registros por cargo. Al implementar esta solución, pudimos aplicar conceptos importantes como clases abstractas, herencia, encapsulamiento, polimorfismo y el manejo de colecciones dinámicas, lo que nos dio como resultado un programa que funciona bien y cumple con lo que necesitábamos.

INDICE

I. INTRODUCCIÓN.....	4
1.1. Objetivo General:.....	4
1.2. Objetivos Especificos:.....	4
II. CONTENIDO DE INVESTIGACION.....	5
2.1 Programación Orientada a Objetos (POO) :.....	5
2.1.1 Clases :.....	6
2.1.2 Encapsulamiento:.....	7
2.1.3 Herencia :.....	7
2.1.4 Poliformismo :.....	8
2.1.5 Uso de Colecciones Dinámicas:.....	8
2.1.6 Cálculo de Salarios :.....	9
2.1.7 Búsqueda del Mayor Salario :.....	10
2.1.8 Ordenamiento por Cargo :.....	10
2.1.9 Manejo de Archivos :.....	11
III. DISCUCIONES.....	12
IV. CONCLUSIONES.....	13
V. RECOMENDACIONES.....	14
VI. REFERENCIAS BIBLIOGRAFICAS.....	15
VII. ANEXOS.....	16

I. INTRODUCCIÓN

La gestión de empleados es algo muy importante en cualquier empresa. Esto se debe a que permite controlar la información sobre el personal, como sus cargos, antigüedad y sueldos. Hacer esto a mano puede llevar a errores y hace difícil acceder rápido a la información que se necesita para tomar decisiones.

En programación, se pueden resolver este tipo de problemas creando sistemas que automaticen el registro y el procesamiento de datos de los trabajadores. La programación orientada a objetos es útil para modelar cosas del mundo real. Permite representar diferentes tipos de empleados y sus características mediante clases y objetos.

En este proyecto, se creó una aplicación en C++ para gestionar a los empleados de una empresa. La aplicación permite registrar a los empleados que trabajan por hora y a los que trabajan de planta. También calcula sus salarios según el cargo que tienen, encuentra a los trabajadores con el salario y la antigüedad más altos, y organiza la información según el cargo. Para hacer esto, se usaron conceptos como el encapsulamiento, la herencia, el polimorfismo y el manejo de estructuras dinámicas de almacenamiento.

1.1. Objetivo General:

- Desarrollar una aplicación en C++ para la gestión de empleados de una empresa mediante el uso de Programación Orientada a Objetos.

1.2. Objetivos Especificos:

- Registrar empleados por hora y empleados de planta.
- Calcular el salario de cada empleado según su cargo.
- Identificar a los empleados con mayor salario y antigüedad.
- Ordenar los empleados de acuerdo con su cargo.
- Aplicar conceptos de herencia, encapsulamiento y polimorfismo.

II. CONTENIDO DE INVESTIGACION

2.1 Programación Orientada a Objetos (POO) :

La Programación Orientada a Objetos, también llamada POO, es una forma de hacer programas. Esto se hace creando clases y objetos. De esta manera, se pueden representar cosas del mundo real dentro de un programa. Esto ayuda a que se pueda reutilizar código, mantener y hacer que las aplicaciones sean más grandes.

La POO se utiliza mucho hoy en día porque hace que los programas sean más organizados y fáciles de entender. En el lenguaje C++, la Programación Orientada a Objetos es algo fundamental. Se utiliza en muchos sistemas, videojuegos, aplicaciones de escritorio y software empresarial.

Características de la POO

- Uso de clases y objetos.
- Reutilización de código mediante herencia.
- Protección de datos con encapsulamiento.
- Flexibilidad mediante polimorfismo.
- Simplificación de sistemas complejos usando abstracción.
- Organización modular del código.
- Facilidad de mantenimiento y escalabilidad.

2.1.1 Clases :

La clase `Empleado` representa la información común de todos los trabajadores, almacenando atributos como nombre, cargo y antigüedad.

Ejemplo en C++:

```
class Empleado {  
private:  
    std::string nombre;  
    std::string cargo;  
    int antigüedad;  
public:  
    virtual double calcularSalario() const = 0;  
    virtual std::string getTipo() const = 0; };
```

Contexto :

Esta clase fue declarada como abstracta debido a la presencia de métodos virtuales puros, los cuales obligan a las clases derivadas a proporcionar su propia implementación.

2.1.2 Encapsulamiento:

El encapsulamiento consiste en proteger los atributos de una clase mediante modificadores de acceso. En la implementación, los datos del empleado fueron declarados como privados.

Ejemplo en C++ :

```
private:
    std::string nombre;
    std::string cargo;
    int antigüedad;
```

El acceso a estos atributos se realiza mediante métodos públicos de consulta y modificación.

```
std::string getNombre() const { return nombre;}
```

2.1.3 Herencia :

La herencia permite reutilizar las características de una clase base. En este programa se crearon dos clases derivadas: `EmpleadoHora` y `EmpleadoPlanta`.

Ejemplo en C++ :

```
class EmpleadoHora : public Empleado {
    // ...
};

class EmpleadoPlanta : public Empleado {
    // ...
};
```

2.1.4 Poliformismo :

El polimorfismo permite que un mismo método tenga diferentes comportamientos según el tipo de objeto.

Ejemplo en C++ :

En la clase base se definió el método virtual puro:

```
virtual double calcularSalario() const = 0;
```

Posteriormente, cada clase derivada implementa su propia versión.
Empleado por hora:

```
double calcularSalario() const override {  
return tarifaHora() * horasTrabajadas; }
```

2.1.5 Uso de Colecciones Dinámicas:

Para almacenar los empleados se utilizó un vector de punteros a la clase base.

Ejemplo en C++ :

```
std::vector<Empleado*> empleados;
```

Contexto :

Esto permitió guardar objetos de distintos tipos dentro de una misma colección y aprovechar el polimorfismo.

2.1.6 Cálculo de Salarios :

El cálculo del salario depende del tipo de empleado. Los empleados de planta reciben un salario semanal basado en una jornada fija de 40 horas, mientras que los empleados por hora reciben un pago proporcional a las horas efectivamente trabajadas.

Las tarifas por hora según el cargo son:

- Gerente: \$100 por hora.
- Supervisor: \$80 por hora.
- Operario: \$50 por hora.

Ejemplo en C++ :

```
double tarifaHora() const {
    if (cargo == "Gerente")
        return 100.0;

    else if (cargo == "Supervisor")
        return 80.0;

    else
        return 50.0; // Operador
}

//EmpleadoPlanta
double calcularSalario() const override {
    return tarifaHora() * 40;
}

//EmpleadoHora
double calcularSalario() const override {
    return tarifaHora() * horasTrabajadas;
}
```

2.1.7 Búsqueda del Mayor Salario :

Para identificar al empleado con la mayor remuneración se recorre toda la lista comparando los salarios calculados.

Ejemplo en C++ :

```
    if (e->calcularSalario() > mayorSalario) {  
        mayorSalario = e->calcularSalario();  
    }
```

2.1.8 Ordenamiento por Cargo :

Los empleados son ordenados utilizando el algoritmo Bubble Sort y una prioridad asignada a cada cargo.

Ejemplo en C++ :

```
    if (empleados[j]->prioridadCargo() >  
        empleados[j + 1]->prioridadCargo()) {  
  
        Empleado* aux = empleados[j];  
        empleados[j] = empleados[j + 1];  
        empleados[j + 1] = aux;  
    }
```

Este procedimiento permite mostrar los empleados agrupados según su cargo dentro de la empresa.

2.1.9 Manejo de Archivos :

Para conservar los resultados obtenidos, se utilizó la biblioteca <fstream>, la cual permite crear y leer archivos de texto.

El programa genera un archivo llamado empleados.txt, donde se almacena la información de todos los empleados, los empleados con mayor salario, los de mayor antigüedad y la lista ordenada por cargo.

Ejemplo en C++ :

```
//Escribir archivos
std::ofstream archivo("empleados.txt");

//Leer archivos
std::ifstream archivo("empleados.txt");
```

III. DISCUCIONES

La implementación que se hizo permitió gestionar a empleados de diferentes tipos. Esto se logró gracias a la Programación Orientada a Objetos. La herencia fue muy útil porque se pudo reutilizar código. El polimorfismo permitió tratar a todos los empleados de la misma manera para hacer cálculos y consultas.

Se utilizó una clase abstracta para organizar mejor el programa. Esto ayudó a diferenciar entre empleados que trabajan por hora y empleados que son de planta. Se crearon funciones para calcular salarios, encontrar al empleado con más antigüedad y ordenar a los empleados según su cargo.

Al final, la solución que se desarrolló cumplió con todos los requisitos. Los resultados fueron correctos y el código quedó claro y fácil de mantener.

IV. CONCLUSIONES

- Se desarrolló correctamente un sistema de gestión de empleados utilizando el lenguaje C++ y los principios de la Programación Orientada a Objetos.
- La aplicación permitió registrar empleados por hora y empleados de planta, así como calcular sus salarios de acuerdo con el cargo desempeñado.
- El uso de herencia y polimorfismo facilitó la organización del código y la implementación de funcionalidades comunes para los distintos tipos de empleados.
- Se cumplieron los requerimientos establecidos en la práctica, permitiendo identificar a los empleados con mayor salario y antigüedad, además de ordenar los registros según el cargo.

V. RECOMENDACIONES

- Implementar validaciones para evitar el ingreso de datos incorrectos o incompletos.
- Incorporar una interfaz gráfica o un menú interactivo para facilitar el uso de la aplicación.
- Utilizar estructuras de datos y algoritmos más eficientes si se requiere gestionar una mayor cantidad de empleados.
- Agregar nuevas funcionalidades, como almacenamiento en archivos o bases de datos, para conservar la información de los empleados de forma permanente.

VI. REFERENCIAS BIBLIOGRAFICAS

- Curso de C++ desde CERO para PRINCIPIANTES (Completo)
<https://www.youtube.com/watch?v=jS6wb263CIM&t=33192s>
- Lippman, S. B., Lajoie, J., & Moo, B. E. (2012). *C++ primer* (5th ed.). Addison-Wesley Professional.
- Weiss, M. A. (2005). *Data structures and algorithm analysis in C++* (3rd ed.). Pearson Addison-Wesley.
- [https://github.com/SB-mike010001/Estructura de Datos Algoritmos 2026/tree/main/week_6/practica_2](https://github.com/SB-mike010001/Estructura_de_Datos_Algoritmos_2026/tree/main/week_6/practica_2)

VII. ANEXOS

```
===== LISTA DE EMPLEADOS =====
```

```
Nombre: Juan Perez  
Cargo: Gerente  
Tipo: Por Hora  
Antigüedad: 8 años  
Horas trabajadas: 20  
Salario: $2000
```

```
-----  
Nombre: Nicoll Torres  
Cargo: Supervisor  
Tipo: De Planta  
Antigüedad: 12 años  
Horas semanales: 40  
Salario: $2000  
-----
```

Figura 1. Lista de empleados

```
===== MAYOR SALARIO =====
```

```
Nombre: Ana Ruiz  
Cargo: Gerente  
Tipo: De Planta  
Antigüedad: 15 años  
Horas semanales: 40  
Salario: $4000  
-----
```

Figura 2. Mayor Salario

```
===== MAYOR TIEMPO EN LA EMPRESA =====
```

```
Nombre: Ana Ruiz  
Cargo: Gerente  
Tipo: De Planta  
Antigüedad: 15 años  
Horas semanales: 40  
Salario: $4000  
-----
```

Figura 3. Mayor Tiempo

```
===== ORDENADOS POR CARGO =====
```

```
(Gerente) Juan Perez - Por Hora  
(Gerente) Ana Ruiz - De Planta  
(Gerente) Miguel Gonzales - Por Hora  
(Operador) Mike Wazowski - Por Hora  
(Operador) Elena Vera - De Planta  
(Operador) Pedro Rojas - Por Hora  
(Operador) Laura Campos - De Planta  
(Supervisor) Nicoll Torres - De Planta  
(Supervisor) Luis Soto - Por Hora  
(Supervisor) Sofia Luna - De Planta
```

Figura 4. Orden por cargo